

<https://medium.com/the-modern-scientist/an-in-depth-look-at-the-transformer-based-models-22e5f5d17b6b>

<https://medium.com/data-science/vision-transformers-explained-a9d07147e4c8#962c>

https://www.youtube.com/watch?v=u-X_nZRsn5M

<https://towardsdatascience.com/multimodal-embeddings-an-introduction-5dc36975966f/>

<https://towardsdatascience.com/clip-intuitively-and-exhaustively-explained-1d02c07dbf40/>

An In-depth Look at the Transformer-based Models.

Executive Summary

The Transformer architecture, introduced in the paper "Attention Is All You Need," revolutionized Natural Language Processing (NLP) by solving the key limitations of older models like RNNs (e.g., slow sequential processing and difficulty with long-range dependencies). Its core innovation is the self-attention mechanism, which allows the model to process all words in a sentence simultaneously and weigh the importance of every word in relation to every other word.

The article then explains that the two most famous Transformer-based models are essentially clever applications of its two main parts:

BERT uses the Encoder part to become a master of understanding language context.

GPT uses the Decoder part to become a master of generating coherent text.

Detailed Breakdown of Key Concepts

1. The Original Transformer Architecture

The article breaks down the original model, designed for machine translation (e.g., English to French), into its essential components:

Input Embedding & Positional Encoding:

Embedding: Words are converted into numerical vectors (embeddings).

Positional Encoding: Since the self-attention mechanism itself doesn't know word order, a vector representing the position of each word (1st, 2nd, 3rd, etc.) is added to its embedding. This gives the model crucial information about the sequence structure.

The Encoder Block:

Purpose: To read the entire input sentence and build a rich, contextual understanding of each word.

Key Component: Multi-Head Self-Attention: This is the heart of the Transformer. It allows each word to "look at" all other words in the input sentence to determine which ones are most relevant to its own meaning. "Multi-head" means it does this from several different perspectives (attention heads) simultaneously to capture different types of relationships (e.g., subject-verb, pronoun-antecedent).

Other Layers: It also includes a simple Feed-Forward Network for further processing and "Add & Norm" layers (residual connections and layer normalization) to help with training stability. An Encoder is typically a stack of several of these blocks.

The Decoder Block:

Purpose: To generate the output sentence (the translation) one word at a time.

Key Components: The Decoder is more complex and has two types of attention:

Masked Multi-Head Self-Attention: This works like the Encoder's self-attention, but it's "masked" to prevent the model from cheating. When generating the 5th word, it can only look at the first 4 words it has already generated, not the future words.

Encoder-Decoder Attention: This is where the Decoder pays attention to the Encoder's output. It allows the word being generated (e.g., the French word) to look at all the English input words to find the most relevant context for an accurate translation.

Final Layers: A final linear layer and a softmax function convert the Decoder's output vector into a probability distribution over the entire vocabulary to select the most likely next word.

2. BERT: The Encoder Master

Core Idea: BERT (Bidirectional Encoder Representations from Transformers) throws away the Decoder and uses only the Transformer's Encoder stack.

Goal: To be pre-trained as a universal language understanding model.

How it Works: Because it uses the Encoder, it is inherently bidirectional. It can see the entire sentence at once, allowing it to build a deep understanding of context from both left and right. It's pre-trained on two tasks:

Masked Language Model (MLM): Guessing randomly hidden words in a sentence (like "fill-in-the-blanks").

Next Sentence Prediction (NSP): Determining if two sentences logically follow each other.

Use Cases: It excels at tasks that require deep contextual understanding, such as sentiment analysis, question answering, and text classification.

3. GPT: The Decoder Master

Core Idea: GPT (Generative Pre-trained Transformer) throws away the Encoder and uses only the Transformer's Decoder stack.

Goal: To be pre-trained as a powerful language generation model.

How it Works: Because it uses the Decoder, it is inherently auto-regressive (generates text one token at a time) and unidirectional. It can only see the words it has already produced (due to the masked self-attention). It's pre-trained on a simple task: predicting the next word in a vast amount of text from the internet.

Use Cases: It excels at tasks that require generating coherent text, such as text completion, summarization, creative writing, and powering chatbots.

Conclusion of the Article

The Transformer is not just a single model but a revolutionary blueprint. By understanding its core Encoder-Decoder structure, you can understand the fundamental difference between the two dominant families of modern language models: those built for deep comprehension (BERT-like) and those built for creative generation (GPT-like).

Vision Transformers Explained

The article explains the **Vision Transformer (ViT)**, a revolutionary model that adapts the highly successful **Transformer architecture** from Natural Language Processing (NLP) to solve computer vision tasks.

The core idea is to **stop treating an image like a 2D grid of pixels** (as Convolutional Neural Networks, or CNNs, do) and instead treat it as a **sequence of smaller image patches**, similar to how a sentence is a sequence of words. By doing this, the model can apply the Transformer's powerful **self-attention mechanism** to find complex, long-range relationships between different parts of an image. While CNNs are excellent at finding local features, ViTs excel at building a **global understanding** of the entire image from the very beginning, leading to state-of-the-art performance, especially when trained on massive datasets.

Detailed Breakdown of Concepts with Explanations

The article meticulously walks through the architecture of a Vision Transformer. Here is a step-by-step explanation of each component:

1. The Problem with CNNs: The "Inductive Bias"

- **How CNNs Work:** CNNs are built on the idea of **locality** and **translation invariance**. They use small filters (kernels) that slide across an image to detect features (like edges, corners, textures) in a small, local neighborhood. As the image passes through deeper layers, these local features are combined to recognize more complex objects.
- **The Inductive Bias:** This built-in assumption that local pixel groups are most important is called an "inductive bias." It's a strength because it makes CNNs very efficient and effective on smaller datasets. However, it can also be a limitation. A CNN has no easy way, in its early layers, to directly relate a patch of pixels on the top-left of an image (e.g., a dog's ear) to a patch on the bottom-right (e.g., its tail). This global context is only built up slowly.
- **The ViT Motivation:** The Transformer's self-attention mechanism was designed to solve exactly this problem in text by finding relationships between words far apart in a sentence. The question the ViT paper answered was: can we use this same mechanism for images?

2. The Vision Transformer Architecture: A Step-by-Step Guide

The article breaks the ViT architecture into six key steps:

- **What it is:** The input image (e.g., 224x224 pixels) is sliced into a grid of smaller, non-overlapping square patches (e.g., 16x16 pixels each).
- **Explanation:** This is the most crucial conceptual leap. It **converts the 2D image into a 1D sequence**. If you have a 224x224 image and use 16x16 patches, you get $(224/16) \times (224/16) = 14 \times 14 = 196$ patches. These 196 patches are then treated as a sequence, just like a sentence with 196 words.
- **What it is:** Each 2D patch (e.g., 16x16x3 pixels) is flattened into a single long vector (e.g., $16 \times 16 \times 3 = 768$ values). This vector is then passed through a standard linear layer to create a "patch embedding" of a fixed size (e.g., $D=768$).
- **Explanation:** This step is analogous to **word embedding in NLP**. It converts the raw pixel data of each patch into a higher-dimensional, meaningful numerical representation that the Transformer can work with. All patches are now represented by vectors of the same length.
- **What it is:** A special, extra learnable vector (the `[class]` token) is added to the beginning of the sequence of patch embeddings. This token doesn't correspond to any specific patch in the image.
- **Explanation:** This is a clever technique borrowed from the famous BERT model in NLP. Instead of trying to aggregate information from all 196 patch embeddings at the end, this `[class]` token acts as a **representative agent**. As it flows through the Transformer layers, the self-attention mechanism allows it to gather and summarize information from all other patches. For the final classification, we only need to look at the final state of this single token.
- **What it is:** A learnable positional embedding is added to each patch embedding (including the `[class]` token). Each positional embedding corresponds to a specific location in the original patch grid (e.g., position (0,0), (0,1), etc.).
- **Explanation:** The self-attention mechanism is **permutation-invariant**—it treats the input sequence as a "bag of patches." It has no inherent sense of order. Without positional embeddings, the model wouldn't know if a patch came from the top-left or the center of the image. By adding these embeddings, we explicitly provide the model with crucial **spatial information**, telling it *where* each patch came from.
- **What it is:** The complete sequence of embeddings (class token + patch embeddings with positional info) is fed into a standard Transformer Encoder. This encoder is a stack of identical layers, each containing:
 1. **Multi-Head Self-Attention:** Allows every patch to "look at" and weigh the importance of every other patch in the image, capturing global context.
 2. **MLP Block (Feed-Forward Network):** Further processes the information for each patch individually.

-
- **Explanation:** This is the core of the ViT. In the self-attention layers, the model learns the relationships between different parts of the image. For example, a patch containing a tire can learn to pay high "attention" to other patches containing tires and a car body, regardless of where they are in the image. This global context is built immediately and refined through each layer of the encoder.
- **What it is:** After the sequence has been processed by the Transformer Encoder, the output vector corresponding to the `[class]` token is taken. This single vector, which now holds a rich, global representation of the entire image, is passed into a small Multi-Layer Perceptron (MLP).
- **Explanation:** This final MLP acts as the classifier. It takes the summarized information from the `[class]` token and maps it to the final class probabilities (e.g., 98% "Dog," 2% "Cat").

Conclusion from the Article: Performance and Implications

- **Data Requirement:** ViTs have less built-in image-specific knowledge than CNNs. As a result, they are **more data-hungry**. On smaller datasets (like ImageNet-1K), they often perform slightly worse than comparable CNNs.
- **State-of-the-Art Performance:** When pre-trained on enormous, private datasets (like Google's JFT-300M with 300 million images), ViTs **surpass the performance of state-of-the-art CNNs** when fine-tuned on smaller, public benchmarks. This proves that with enough data, they can learn the patterns of vision from scratch even more effectively than CNNs.
- **The Future is Likely Hybrid:** The article concludes by noting the success of hybrid models that combine the strengths of both architectures—using a CNN backbone to efficiently extract local features and then feeding them into a Transformer to model global relationships.

Multimodal Embeddings: An Introduction.

Executive Summary

The article introduces multimodal embeddings, a powerful concept in AI for creating a unified, shared vector space where different types of data (modalities)—like text, images, and audio—can be represented and compared.

The fundamental goal is to achieve semantic alignment. This means that a concept, regardless of its form, should map to the same (or a very similar) point in this shared space. For example,

the text "a photo of a happy dog" should have a vector representation that is very close to the vector for an actual image of a happy dog.

This is achieved by training separate "encoder" models for each modality (e.g., a Transformer for text, a CNN or ViT for images) using a contrastive learning approach. By learning to match correct pairs (text + corresponding image) and push apart incorrect pairs, the models learn a shared "language" of vectors. This technology is the foundation for revolutionary applications like text-to-image search (Google Images), zero-shot classification (CLIP), and generative AI (DALL-E, Midjourney).

Detailed Breakdown of Key Concepts

The article builds the concept of multimodal embeddings from the ground up. Here is a step-by-step explanation of its main points.

1. The Foundation: Unimodal Embeddings

What they are: A unimodal embedding is a numerical vector that represents a single piece of data from one modality.

The Key Property: These embeddings are semantically meaningful. In the vector space, items with similar meanings are located close to each other.

Text Example (Word2Vec, BERT): The vector for "king" is close to "queen," and the vector for "walking" is close to "running."

Image Example (ResNet, ViT): The embedding for an image of a German Shepherd is closer to an image of a Golden Retriever than to an image of a car.

The Limitation: These embeddings live in separate, incompatible spaces. You cannot directly compare a text embedding with an image embedding.

2. The Leap: What are Multimodal Embeddings?

The Core Idea: To create a single, "universal" vector space that can represent information from multiple modalities simultaneously. It acts like a shared dictionary or a universal translator.

The Goal (Semantic Alignment): In this shared space, the vector for a text description should be mathematically close to the vector of an image that matches that description. The model learns to map concepts, not just raw data.

3. How are Multimodal Embeddings Created?

The article explains the training architecture and process, which typically involves two main components:

You need a specialized model to process each type of data and convert it into an initial embedding.

Text Encoder: Typically a Transformer-based model (like BERT) that reads a sequence of words and outputs a single vector representing the entire text.

Image Encoder: Typically a Convolutional Neural Network (CNN like ResNet) or a Vision Transformer (ViT) that processes an image and outputs a single vector.

This is the "secret sauce" that aligns the two separate vector spaces. The most famous method, used by OpenAI's CLIP, is Contrastive Language-Image Pre-training.

The Dataset: A massive dataset of (image, text) pairs. For CLIP, this was 400 million pairs scraped from the internet.

The Process: The model is shown a batch of these pairs. For any given image:

Positive Pair: Its correct, corresponding text caption. The model's goal is to maximize the similarity (e.g., the cosine similarity) between the image embedding and the text embedding for this pair. It tries to pull them closer together in the vector space.

Negative Pairs: All other text captions in the batch. The model's goal is to minimize the similarity between the image embedding and these incorrect text embeddings. It tries to push them far apart.

The Outcome: By repeating this process millions of times, the text encoder and image encoder learn to produce aligned embeddings. They are forced to understand the high-level semantic concepts that bridge the two modalities to succeed at this matching game.

4. The Star Example: OpenAI's CLIP

What it is: The quintessential example of a model that produces powerful multimodal embeddings for text and images.

The Breakthrough: Zero-Shot Classification. Because CLIP understands the relationship between images and descriptive text so well, it can perform image classification on tasks it was never explicitly trained on.

How it works: To classify an image of a dog, you give the model the image and a set of text prompts like "a photo of a dog," "a photo of a cat," "a photo of a car." The model converts the image and all the text prompts into embeddings. It then calculates which text embedding is closest to the image embedding. The winning text prompt becomes the classification label. This is incredibly flexible and powerful.

5. Key Applications Powered by Multimodal Embeddings

The article concludes by highlighting the groundbreaking applications that this technology enables:

Cross-Modal Search: The most intuitive use. You can search a database of images using a text query (or vice-versa). The system finds the images whose embeddings are closest to your text query's embedding.

Zero-Shot Image Classification: As explained with CLIP, classifying images using arbitrary text labels without needing a dedicated training dataset for those labels.

Image Captioning: By providing an image, the model can find the most semantically similar text concepts in its learned space to generate a descriptive caption.

Text-to-Image Generation (DALL-E, Midjourney, Stable Diffusion): This is a more advanced application where the text embedding acts as a guiding signal for a generative model. The model tries to create an image whose embedding gets progressively closer to the target text embedding, resulting in an image that matches the prompt.

Multimodal Reasoning: Answering complex questions about the content of an image, which requires a deep, shared understanding of both visual and textual concepts.

CLIP, Intuitively and Exhaustively Explained

The article "CLIP, Intuitively and Exhaustively Explained" demystifies OpenAI's groundbreaking CLIP (Contrastive Language-Image Pre-training) model. It explains that CLIP's "magic" isn't a single complex invention, but rather the successful combination of three existing ideas at an unprecedented scale:

A Massive, "Wild" Dataset: Instead of a clean, human-labeled dataset, CLIP was trained on 400 million (image, text) pairs scraped directly from the internet. This provides enormous diversity and scale.

A Simple Dual-Encoder Architecture: It uses two separate models—an Image Encoder (like a ViT) and a Text Encoder (a Transformer)—that learn to map their respective inputs into a shared, multimodal embedding space.

A Powerful Training Objective (Contrastive Learning): The model learns by playing a giant "matching game." For a batch of image-text pairs, it is trained to maximize the similarity between the correct pairs while minimizing the similarity between all incorrect combinations.

The result is a model that acts like a Rosetta Stone for images and text. It understands the semantic relationship between visual and textual concepts so well that it can perform its most famous feat: zero-shot image classification, allowing it to classify images into categories it has never been explicitly trained on, simply by comparing the image to a text description of the categories.

Detailed Breakdown of Key Concepts

The article breaks down CLIP into its core components and processes, which can be summarized as follows:

1. The Problem CLIP Solves: The Limits of Traditional Supervised Learning

High Cost & Limited Scope: Standard image classifiers require massive, manually-labeled datasets (like ImageNet). This is expensive, time-consuming, and restricts the model to a pre-defined, fixed set of categories. A model trained on ImageNet knows 1,000 classes and nothing else.

Poor Generalization: These models are often brittle and don't generalize well to tasks or visual styles outside their training data.

CLIP's Approach: Instead of learning from a fixed set of labels, CLIP learns the rich, nuanced relationship between images and the "free-form" text used to describe them online. This allows it to learn a much broader range of visual concepts.

2. The Three Pillars of CLIP's Success

The article structures its explanation around the three key ingredients that make CLIP work.

Source: A new dataset of 400 million (image, alt-text) pairs collected from the public internet.

"Weak Supervision": The alt-text captions are not always perfect or purely descriptive. They are "noisy." However, the sheer scale of the dataset compensates for this noise.

The Advantage: This approach provides a nearly limitless and incredibly diverse source of supervision, covering a vast array of objects, scenes, actions, and styles far beyond what curated datasets can offer.

The model consists of two independent but co-trained components:

Image Encoder: A model that takes an image as input and outputs a single vector (embedding). The CLIP paper experimented with both ResNet and, more successfully, the Vision Transformer (ViT).

Text Encoder: A standard Transformer model that takes text as input and outputs a single vector (embedding).

The Goal: To train these two encoders so that the image embedding for a picture of a dog is mathematically "close" to the text embedding for the phrase "a picture of a dog" in a shared vector space.

This is the core mechanism that aligns the two encoders. The article explains it with a clear, step-by-step example:

Create a Batch: Take a batch of N (image, text) pairs from the dataset. For example, $N=3$.

(Image_1, "a photo of a cat")

(Image_2, "a photo of a car")

(Image_3, "a photo of a dog")

Encode Everything:

Pass all N images through the Image Encoder to get N image embeddings (I_1, I_2, I_3).

Pass all N texts through the Text Encoder to get N text embeddings (T_1, T_2, T_3).

Calculate Similarities: Compute the cosine similarity between every image embedding and every text embedding. This results in an $N \times N$ matrix.

	T_1 (cat)	T_2 (car)	T_3 (dog)
I_1 (cat)	High	Low	Low
I_2 (car)	Low	High	Low
I_3 (dog)	Low	Low	High

Define the Loss:

Positive Pairs: The correct matches are along the diagonal of the matrix (I_1 with T_1 , I_2 with T_2 , etc.). The model's objective is to maximize these similarity scores.

Negative Pairs: All other cells in the matrix are incorrect pairs (I_1 with T_2 , I_3 with T_1 , etc.). The objective is to minimize these similarity scores.

Learn and Repeat: By optimizing this contrastive loss, the encoders are forced to create a shared space where concepts are aligned. The Image Encoder learns what visual features correspond to the concepts the Text Encoder learns from words.

3. The Payoff: Zero-Shot Prediction

This is how the trained CLIP model is actually used. It's a different process from training.

The Task: You have an image you want to classify (e.g., a picture of a dog).

Create Text Prompts: You define your potential classes as descriptive text sentences. This is a crucial step. Instead of just "dog," you use "a photo of a dog," "a photo of a cat," "a photo of an airplane." This matches the style of the training data.

Encode Inputs:

The image is passed through the trained Image Encoder to get a single image embedding.

Each of your text prompts is passed through the trained Text Encoder to get a text embedding for each class.

Find the Best Match: The model calculates the cosine similarity between the single image embedding and all the text embeddings.

Make the Prediction: The text prompt that results in the highest similarity score is the model's prediction.

The revolutionary part is that this requires no fine-tuning or gradient updates. The model can classify images into any set of categories you can describe with text, making it incredibly flexible and powerful.